

Transferleistung Theorie/Praxis 6

Matrikelnummer:	12315
Freigegebenes Thema:	Agile Dokumentation: Wie viel Dokumentation ist sinnvoll?
Studiengang, Zenturie:	Angewandte Informatik, A22a

Inhaltsverzeichnis

Abkürzungen	II
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung und Forschungsfrage	1
2 Theoretische Grundlagen von Dokumentation in agilen Projekten	3
2.1 Grundlagen agiler Softwareentwicklung	3
2.2 Merkmale guter Projektdokumentation	3
2.3 Vorteile guter Projektdokumentation	4
2.4 Die Rolle von Dokumentationsaufwand in agilen Projekten	5
3 Bestimmung eines sinnvollen Dokumentationsumfangs	7
3.1 Bewertungskriterien eines sinnvollen Umfangs	7
3.1.1 Zeitlicher und organisatorischer Umgang mit Dokumentation	7
4 Praxisbezug	9
4.1 Projektbeschreibung von Finrep	9
4.2 Bewertung bestehender Dokumentation und Dokumentationspraktiken	9
4.3 Handlungsempfehlung	10
5 Fazit	12
5.1 Beantwortung der Forschungsfrage	12
5.2 Ausblick	12
Bibliographie	III

Abkürzungen

1 Einleitung

1.1 Problemstellung

Agile Softwareprojekte sind auf kurze Entwicklungszyklen, frühe nutzbare Ergebnisse und die flexible Reaktion auf veränderte Anforderungen ausgerichtet. Dokumentation wird in diesem Umfeld teilweise als Hindernis wahrgenommen, da ihre Erstellung Zeit bindet, die nicht unmittelbar in neue Funktionalität einfließt. Gleichzeitig ist sie erforderlich, um Projektwissen zu sichern, die Einarbeitung neuer Mitarbeitender zu erleichtern und die langfristige Wartbarkeit eines Systems zu unterstützen.

Der daraus entstehende Zielkonflikt zeigt sich besonders in langlebigen Projekten. Unter Zeitdruck wird Dokumentation häufig unvollständig gepflegt, während bestehende Inhalte über Jahre wachsen und dadurch unübersichtlich oder veraltet werden können. Dies erhöht den Abstimmungsaufwand, erschwert Änderungen und verstärkt die Abhängigkeit von einzelnen Wissensträgern. Eine möglichst vollständige Dokumentation ist jedoch ebenfalls nicht zweckmäßig, da sich technische und fachliche Inhalte in agilen Projekten regelmäßig verändern und mit entsprechendem Aufwand aktualisiert werden müssten.

Die zentrale Herausforderung besteht daher darin, einen angemessenen Dokumentationsumfang zu bestimmen. Dokumentation muss die für Entwicklung, Wartung und Betrieb erforderlichen Informationen bereitstellen, darf jedoch weder zum Selbstzweck werden noch den Entwicklungsfortschritt unverhältnismäßig beeinträchtigen. Entscheidend ist folglich nicht die größtmögliche Menge, sondern die Auswahl relevanter, verständlicher, auffindbarer und dauerhaft pflegbarer Inhalte.

1.2 Zielsetzung und Forschungsfrage

Ziel dieser Transferleistung ist es, den sinnvollen Umfang von Dokumentation in agilen Softwareprojekten theoretisch zu untersuchen und auf das Praxisbeispiel Finrep anzuwenden. Dazu werden zunächst die Grundlagen agiler Softwareentwicklung sowie Merkmale, Vorteile und Aufwände guter Projektdokumentation betrachtet. Darauf aufbauend werden Kriterien entwickelt, mit denen sich ein angemessener Dokumentationsumfang bewerten lässt.

Im Praxisteil werden die vorhandene Dokumentation und die erkennbaren Dokumentationspraktiken des Projekts Finrep anhand dieser Kriterien eingeordnet. Aus den festgestellten Stärken und Schwächen werden anschließend Handlungsempfehlungen abgeleitet. Die zentrale Forschungsfrage lautet:

Wie viel Dokumentation ist in agilen Softwareprojekten sinnvoll, um Projektwissen nachhaltig zu sichern, ohne den Entwicklungsfortschritt unverhältnismäßig zu beeinträchtigen?

Der Fokus liegt damit nicht auf einem allgemeingültigen Mindest- oder Höchstumfang, sondern auf der Frage, welche Dokumentation im jeweiligen Projektkontext tatsächlich hilfreich, aktuell und nutzbar ist.

2 Theoretische Grundlagen von Dokumentation in agilen Projekten

2.1 Grundlagen agiler Softwareentwicklung

Agile Softwareentwicklung bezeichnet einen iterativen und inkrementellen Ansatz, der kurze Entwicklungszyklen, frühes Feedback, enge Zusammenarbeit mit Stakeholdern und eine hohe Anpassungsfähigkeit gegenüber Veränderungen betont [1], [2]. Anders als in stark planungsgetriebenen Modellen werden Anforderungen, Entwurf, Implementierung und Test nicht ausschließlich als nacheinander ablaufende Phasen verstanden. Stattdessen entstehen früh lauffähige Teilergebnisse, die regelmäßig überprüft und weiterentwickelt werden. Agile Methoden gelten daher als leichtgewichtige Vorgehensweisen für dynamische Projektumfelder, in denen Anforderungen zu Beginn nicht vollständig feststehen [1], [2].

Agilität bedeutet nicht lediglich Schnelligkeit, sondern vor allem den systematischen Umgang mit Unsicherheit. Fachliche Anforderungen, technische Bedingungen und organisatorische Prioritäten können sich während des Projektverlaufs verändern [3]. Kurze Iterationen, Reviews, Retrospektiven und die kontinuierliche Abstimmung zwischen Entwicklung und Fachseite ermöglichen es, diese Veränderungen früh zu erkennen und in die weitere Planung einzubeziehen. Agile Entwicklung ist somit nicht mit ungeplantem Arbeiten gleichzusetzen, sondern verankert Anpassungsfähigkeit bewusst im Prozess [3].

Empirische Untersuchungen zeigen, dass Teamorientierung und ein funktionierender Informationsfluss wesentliche Voraussetzungen erfolgreicher agiler Projektarbeit sind [4]. Direkte Kommunikation reduziert zwar formale Übergaben, kann eine dauerhafte und strukturierte Sicherung wichtiger Informationen jedoch nicht vollständig ersetzen [4].

Die Wirksamkeit agiler Methoden hängt zudem vom Projektkontext ab. Projektgröße, Kritikalität, Teamstruktur, Lebensdauer und organisatorisches Umfeld beeinflussen, wie stark Prozesse formalisiert und Wissen dokumentiert werden muss [5]. Agile Vorgehensweisen geben daher keinen vollständigen Verzicht auf Dokumentation vor, sondern verlangen eine bewusste Abwägung zwischen Flexibilität und notwendiger Nachvollziehbarkeit [5].

2.2 Merkmale guter Projektdokumentation

Ein zentrales Qualitätsmerkmal von Projektdokumentation ist die Zweckorientierung [6]. Jedes Dokument sollte eine konkrete Zielgruppe und Aufgabe besitzen, beispielsweise die Erklärung einer Architekturentscheidung, die Beschreibung einer Schnittstelle, die Unterstützung des Betriebs oder die Einarbeitung neuer Mitarbeitender [6]. Untersuchungen zur Nutzung von Softwaredokumentation zeigen, dass Inhalte langfristig vor allem dann nützlich bleiben, wenn sie relevant und mit vertretbarem Aufwand pflegbar sind [6]. Eine umfangreiche Informationssammlung ohne klaren

Verwendungszweck erzeugt dagegen Pflegeaufwand, ohne die Projektarbeit entsprechend zu unterstützen [6].

Ebenso wichtig ist Aktualität [7]. Veraltete Dokumentation kann falsche Annahmen erzeugen und dadurch Fehlentscheidungen begünstigen. Aktualität bedeutet jedoch nicht, dass jedes Detail jederzeit vollständig beschrieben sein muss [7]. Vorrangig aktuell zu halten sind Informationen, die für Entwicklung, Wartung, Betrieb oder wiederkehrende Entscheidungen benötigt werden [7]. Die praktische Nützlichkeit hängt sowohl von der Relevanz der Inhalte als auch von geeigneten Werkzeugen und Pflegeprozessen ab [7].

Dokumentation muss außerdem verständlich und auffindbar sein [8]. Inhalte sollten auch von Personen nachvollzogen werden können, die nicht an der ursprünglichen Entscheidung beteiligt waren. Klare Sprache, konsistente Benennungen, eine geeignete Granularität und eine übersichtliche Struktur sind dafür ebenso wichtig wie fachliche Korrektheit [8]. Selbst inhaltlich vollständige Dokumentation verliert einen großen Teil ihres Nutzens, wenn relevante Informationen nur mit hohem Suchaufwand gefunden werden können [8].

Ein weiteres Merkmal ist Nachvollziehbarkeit [9], [10]. Gute Dokumentation hält nicht nur fest, was umgesetzt wurde, sondern erläutert bei wesentlichen Entscheidungen auch die Gründe, Ausgangsbedingungen und gegebenenfalls verworfenen Alternativen. Bei Anforderungen ist zudem eine Verbindung zwischen fachlichem Bedarf, Umsetzung und Test sinnvoll [9], [10]. Agile Anforderungsarbeit kann diese Informationen leichtgewichtig durch User Stories, Backlogs, Akzeptanzkriterien und Testfälle sichern [9], [10].

Schließlich muss Dokumentation wartbar sein [11]. Inhalte, die nur mit hohem Aufwand aktualisiert werden können oder dieselben Informationen mehrfach enthalten, werden im Projektalltag schnell vernachlässigt [11]. Dokumentation sollte deshalb möglichst nah am Arbeitsprozess entstehen, klar verantwortet und auf notwendige Ergänzungen zu Code, Tests und Tickets beschränkt werden [11]. Für die spätere Wartung sind insbesondere Informationen zu Systemverhalten, Struktur und Änderungsfolgen relevant [11].

Gute Projektdokumentation ist somit zweckmäßig, aktuell, verständlich, auffindbar, nachvollziehbar und mit angemessenem Aufwand pflegbar.

2.3 Vorteile guter Projektdokumentation

Ein wesentlicher Vorteil guter Dokumentation liegt im Wissenstransfer [12]. Softwareentwicklung ist wissensintensiv, wobei zentrale Informationen häufig in fachlichen Abwägungen, technischen Entscheidungen und dem Erfahrungswissen einzelner Personen liegen [12]. Bleibt dieses Wissen ausschließlich implizit, entstehen Risiken bei Krankheit, Rollenwechsel, Fluktuation oder wachsender Teamgröße. Auch in agilen Teams bleibt Dokumentation deshalb notwendig, um Wissen dauerhaft verfügbar zu machen und Informationsverluste zu begrenzen [12].

Dokumentation unterstützt außerdem die Wartung und Weiterentwicklung langlebiger Systeme [6], [11]. Änderungen werden häufig von Personen durchgeführt, die nicht an der ursprünglichen Implementierung beteiligt waren. Für sie sind Übersichten zu Architektur, Systemverhalten, Abhängigkeiten und Änderungsfolgen besonders relevant [11]. Aufgabenbezogene Dokumentation erleichtert die Orientierung und verringert den Aufwand, Zusammenhänge ausschließlich aus dem Quellcode rekonstruieren zu müssen [6].

Gute Dokumentation verbessert die Kommunikation zwischen Rollen und Stakeholdern [13]. Direkte Gespräche ermöglichen schnelle Abstimmung, sichern deren Ergebnisse jedoch nicht dauerhaft [13]. Werden fachliche Annahmen, Entscheidungen oder technische Einschränkungen nicht festgehalten, können sie später unterschiedlich interpretiert werden. Kommunikationslücken können insbesondere in größeren Projekten zu Qualitätsproblemen, verfehlten Erwartungen und zusätzlichem Aufwand führen [13]. Dokumentation schafft hier ein gemeinsames Referenzsystem [13].

Auch die Qualitätssicherung profitiert von dokumentierten Anforderungen, Akzeptanzkriterien, Testfällen und bekannten Einschränkungen [14], [15]. Informelle Kommunikation bietet Flexibilität, kann jedoch zu unklaren oder widersprüchlichen Anforderungen führen [14]. Leichtgewichtige und laufend priorisierte Anforderungsartefakte verbinden Nachvollziehbarkeit mit der notwendigen Veränderbarkeit [15].

Mit zunehmender Projektgröße und Lebensdauer steigt schließlich die organisatorische Bedeutung von Dokumentation [16]. Wissen muss über Team- und Rollengrenzen hinweg verfügbar bleiben, besonders bei regulatorischen, betrieblichen oder vertraglichen Anforderungen [16]. Untersuchungen zur großskaligen agilen Entwicklung zeigen, dass die Koordination von Anforderungen und Wissen in größeren Organisationen eine besondere Herausforderung darstellt [16]. Dokumentation reduziert dabei die Abhängigkeit von informellen Kommunikationswegen und einzelnen Personen [16].

2.4 Die Rolle von Dokumentationsaufwand in agilen Projekten

Empirische Untersuchungen zeigen, dass interne Dokumentation in agilen Teams gleichzeitig als notwendig und als häufig vernachlässigt wahrgenommen wird [12]. In kleinen, stabilen Teams kann direkte Kommunikation einen Teil der formalen Dokumentation ersetzen [12]. Mit wachsender Systemkomplexität, längerer Laufzeit, personellen Veränderungen und steigender Anzahl beteiligter Personen nimmt der Bedarf an dauerhaft verfügbarem Wissen jedoch zu [12]. Der angemessene Aufwand muss deshalb kontextabhängig bestimmt werden [12].

Agile Dokumentation ist nicht mit möglichst wenig Dokumentation gleichzusetzen. Entscheidend ist der Nutzen der einzelnen Artefakte [17]. Die Literatur zeigt, dass in der Praxis unterschiedliche Auffassungen darüber bestehen, welche Inhalte in agilen Projekten festgehalten werden sollten [17]. Dokumentation muss daher bewusst gestaltet und in den Entwicklungsprozess integriert werden, statt lediglich nebenbei oder erst nach Abschluss einer Aufgabe zu entstehen [17].

Wichtige Einflussfaktoren sind Teamgröße und Fluktuation, fachliche und technische Komplexität, Kritikalität, externe Schnittstellen, regulatorische Anforderungen und die erwartete Lebensdauer des Systems [5]. In kleinen Projekten mit stabiler Teamzusammensetzung kann ein geringer formaler Umfang genügen [5]. Bei großen, langlebigen oder kritischen Systemen ist eine stärkere Dokumentation erforderlich, weil Wissen über längere Zeit und für mehr Beteiligte verfügbar bleiben muss [5]. Zu viel Formalisierung kann die Anpassungsfähigkeit einschränken, während zu wenig Struktur Risiken für Qualität, Koordination und Wartbarkeit erzeugt [5].

Dokumentation sollte deshalb inkrementell und arbeitsnah entstehen. Geeignet sind unter anderem User Stories, Akzeptanzkriterien, kurze Architekturentscheidungen, Schnittstellenbeschreibungen, Testdokumentation, Betriebsanleitungen und Onboarding-Hinweise [10], [15]. Diese Artefakte müssen nicht umfangreich sein, sollten aber Informationen sichern, die ohne Dokumentation nur schwer rekonstruierbar wären [10], [15]. Leichtgewichtige und iterativ gepflegte Artefakte können Flexibilität und Nachvollziehbarkeit miteinander verbinden [10], [15].

Zusammenfassend ist der Aufwand damit als Investition in die zukünftige Entwicklungsfähigkeit zu betrachten. Kurzfristig kann weniger Dokumentation Zeit sparen. Langfristig können jedoch längere Einarbeitung, wiederholte Rückfragen, Fehlinterpretationen, technische Schulden und erschwerte Wartung höhere Kosten verursachen. Sinnvolle agile Dokumentation vermeidet deshalb unnötige oder redundante Inhalte, hält aber das für Verständnis, Weiterentwicklung und Betrieb wesentliche Wissen fest.

3 Bestimmung eines sinnvollen Dokumentationsumfangs

3.1 Bewertungskriterien eines sinnvollen Umfangs

Aus den theoretischen Grundlagen lassen sich fünf Kriterien zur Bewertung des Dokumentationsumfangs ableiten.

Das erste Kriterium ist die Zweckmäßigkeit. Jedes Artefakt sollte einen konkreten Nutzen für Einarbeitung, Wartung, Fehleranalyse, Entscheidungsnachverfolgung oder Betrieb erfüllen. Dokumente ohne klare Zielgruppe oder Verwendung erzeugen zusätzlichen Pflegeaufwand und sollten reduziert, zusammengeführt oder archiviert werden.

Das zweite Kriterium ist die Relevanz der Inhalte. Vorrangig zu dokumentieren sind Informationen, die für das Systemverständnis notwendig und nicht ohne Weiteres aus Code, Tests oder Tickets erkennbar sind. Dazu gehören insbesondere fachliche Besonderheiten, zentrale Architekturentscheidungen, externe Schnittstellen, Betriebsabläufe sowie Abhängigkeiten zu anderen Systemen. Allgemein erkennbare oder automatisch erzeugbare Informationen müssen dagegen nicht mehrfach beschrieben werden.

Das dritte Kriterium ist die Aktualisierbarkeit. Der Umfang ist nur dann angemessen, wenn das Team die Inhalte im Projektalltag zuverlässig pflegen kann. Eine kleinere, aber aktuelle Dokumentationsbasis ist wertvoller als eine umfangreiche Sammlung veralteter Informationen. Häufig veränderliche Angaben sollten deshalb automatisiert erzeugt, zentral konfiguriert oder durch stabile Verweise ersetzt werden.

Das vierte Kriterium umfasst Auffindbarkeit und Verständlichkeit. Dokumentation erfüllt ihren Zweck nur, wenn relevante Inhalte schnell gefunden und von der jeweiligen Zielgruppe verstanden werden können. Einheitliche Benennungen, eine zentrale Navigation, klare Strukturen und eine passende Sprache sind daher wesentliche Bestandteile eines sinnvollen Umfangs. Verteilte Dubletten erhöhen dagegen sowohl Such- als auch Pflegeaufwand.

Das fünfte Kriterium ist der Projektkontext. Der erforderliche Umfang steigt mit Komplexität, Kritikalität, Lebensdauer, Teamgröße, regulatorischen Anforderungen und der Bedeutung externer Schnittstellen. Eine Dokumentation ist somit sinnvoll umfangreich, wenn sie die für Entwicklung, Wartung und Betrieb wesentlichen Informationen zuverlässig bereitstellt, ohne den agilen Prozess durch unnötige oder schwer pflegbare Inhalte zu belasten.

3.1.1 Zeitlicher und organisatorischer Umgang mit Dokumentation

Dokumentation sollte parallel zur Entwicklung entstehen und bei relevanten Änderungen unmittelbar angepasst werden [10], [15]. Leichtgewichtige, iterativ gepflegte Artefakte sichern notwendige Informationen, ohne die Anpassungsfähigkeit des Projekts unnötig einzuschränken [10], [15].

Organisatorisch sollte die Dokumentationspflege Bestandteil der jeweiligen Entwicklungsaufgabe sein [12]. Da sie trotz ihres wahrgenommenen Nutzens im Projektalltag häufig vernachlässigt wird, sind klare Zuständigkeiten und regelmäßige Überprüfungen erforderlich [12]. Dies kann beispielsweise durch Dokumentationsanforderungen in der Definition of Done, Seitenverantwortliche sowie feste Review- und Archivierungsintervalle erfolgen [12].

Ein allgemeingültiger fester Zeitanteil ist nicht zweckmäßig. Der erforderliche Aufwand hängt von Komplexität, Kritikalität, Lebensdauer und Veränderungshäufigkeit ab [5], [17]. Er sollte daher bedarfs- und risikoorientiert festgelegt werden, sodass ausreichende Nachvollziehbarkeit entsteht, ohne den Entwicklungsfortschritt unverhältnismäßig zu beeinträchtigen [5], [17].

4 Praxisbezug

4.1 Projektbeschreibung von Finrep

IDL Finrep ist eine Windows-Desktop-Anwendung zur Erstellung und Validierung regulatorischer Finanzmeldungen. Sie unterstützt Banken bei ihren Meldepflichten gegenüber europäischen Zentralbanken und verarbeitet insbesondere den Berichtsstandard FINREP. Neben Import und Zuordnung von Finanzdaten umfasst die Anwendung Validierungs-, Auswertungs- und Exportfunktionen. Die lange Projektlaufzeit, regelmäßige Taxonomieänderungen und regulatorische Anforderungen machen Finrep zu einem fachlich komplexen und kontinuierlich weiterentwickelten System.

4.2 Bewertung bestehender Dokumentation und Dokumentationspraktiken

Die Dokumentationslandschaft von Finrep ist umfangreich und besteht aus unterschiedlichen Artefakten. Im Repository befindet sich eine ausführliche README-Datei. In Confluence stehen unter anderem eine Architekturübersicht, Benutzer- und Installationsanleitungen, FAQ, Glossar, Berechtigungsmatrizen sowie Dokumentationen zu Taxonomien und DPM 2.0 zur Verfügung. Sprint-Reviews und strukturierte Jira-Vorgänge ergänzen diese Inhalte durch Informationen zu Fehlern, Anforderungen, Tests und technischen Aufgaben.

Hinsichtlich der Zweckmäßigkeit ist die Dokumentation überwiegend positiv zu bewerten. Die meisten Artefakte besitzen einen erkennbaren Verwendungszweck und richten sich an konkrete Zielgruppen. Benutzer- und Installationsleitfäden unterstützen Anwender und Betrieb, während Architektur- und Entwicklungsdokumentationen der Einarbeitung und Wartung dienen. Jira-Vorgänge sichern Anforderungen, Fehlerbilder und Umsetzungsschritte. Einschränkend bestehen einzelne Überschneidungen, etwa mehrfach vorhandene Feature-Matrizen, sowie weiterhin auffindbare Altseiten, die teilweise lediglich als nicht mehr verwendet gekennzeichnet sind.

Auch die Relevanz der Inhalte ist grundsätzlich hoch. Architekturübersicht und Jira-Vorgänge halten fachliche Hintergründe, kundenspezifische Anforderungen, technische Einschränkungen und Begründungen fest, die nicht unmittelbar aus dem Quellcode hervorgehen. Dadurch bleibt ein wesentlicher Teil des Kontextwissens für die Weiterentwicklung verfügbar.

Defizite zeigen sich vor allem innerhalb des Quellcodes. Zentrale Geschäftsobjekte enthalten teilweise keine erläuternden Kommentare. Bei Datenzugriffsmethoden mit eingebetteten SQL-Anweisungen fehlen mitunter Hinweise zu Zweck, betroffenen Tabellen und fachlichen Prozessen. Auch komplexe Berechnungs- und Validierungsabläufe sind teilweise stark vom Erfahrungswissen einzelner Teammitglieder abhängig. Der zeitweise Verlust von Detailwissen zur Einbindung einer Berechnungseingabe während einer Taxonomieumstellung verdeutlicht das daraus entstehende Risiko.

Die Aktualisierbarkeit ist differenziert zu bewerten. Positiv sind die hohe Pflegeaktivität, zahlreiche Überarbeitungen im Jahr 2026 und die fortlaufenden Sprint-Reviews. Der Ansatz der Concurrent Documentation unterstützt zudem die parallele Erstellung von Software und Dokumentation. Gleichzeitig enthalten einzelne Dokumente volatile Angaben wie Servernamen, absolute Netzwerkpfade, Versionsnummern oder Taxonomiebezeichnungen. Solche Inhalte erhöhen den Pflegeaufwand und können schnell veralten. Die Architekturübersicht ist lediglich eine datierte Momentaufnahme; ohne regelmäßige oder automatisierte Erzeugung besteht ein Drift-Risiko. Offene Platzhalter für Abbildungen zeigen außerdem, dass parallel begonnene Dokumentation nicht immer vollständig abgeschlossen wird.

Bei Auffindbarkeit und Verständlichkeit bestehen ebenfalls Stärken und Schwächen. Ein zentraler Confluence-Bereich, Inhaltsverzeichnisse, Glossar, Jira-Komponenten und Verknüpfungen zum Repository unterstützen die Orientierung. Zugleich sind Inhalte auf mehrere Bereiche und teilweise persönliche Ablageorte verteilt. Sprache und Benennung wechseln zwischen Deutsch und Englisch, teilweise innerhalb desselben Dokuments. Dubletten, uneinheitliche Begriffe und schwer darstellbare Diagramme erschweren dadurch die Suche und das Verständnis.

Unter Berücksichtigung des Projektkontexts ist der vorhandene Umfang insgesamt angemessen. Finrep ist langlebig, fachlich komplex, regulatorisch relevant und mit mehreren externen Systemen verbunden. Ein lediglich minimaler Dokumentationsumfang würde diesen Bedingungen nicht gerecht. Der Verbesserungsbedarf ist daher weniger quantitativ als qualitativ: Wissen muss an kritischen Stellen besser im Code gesichert, technische Übersichten gegen Veraltung geschützt und die Dokumentationslandschaft konsistenter strukturiert werden.

4.3 Handlungsempfehlung

Aus der Bewertung ergeben sich mehrere bedarfsorientierte Maßnahmen. Der Aufwand sollte nicht pauschal erhöht, sondern auf kritische, langlebige und schwer rekonstruierbare Bereiche konzentriert werden.

Zunächst sollte die In-Code-Dokumentation gezielt ergänzt werden. Besonders relevant sind zentrale Geschäftsobjekte, die Berechnungs- und Validierungslogik sowie Datenzugriffsmethoden mit eingebettetem SQL. Kurze Kommentare sollten Zweck, Parameter, fachliche Besonderheiten und betroffene Tabellen erläutern. Eine flächendeckende Kommentierung ist nicht erforderlich; entscheidend sind komplexe Stellen, deren Bedeutung nicht direkt aus dem Code hervorgeht.

Technische Momentaufnahmen sollten besser gegen Veraltung abgesichert werden. Die Architekturübersicht kann automatisiert in der Continuous-Integration-Pipeline erzeugt und mit Generierungsdatum sowie Quellstand versehen werden. Veränderliche Servernamen, Pfade, Versions- oder Taxonomiesuffixe sollten möglichst in zentralen Konfigurationen gepflegt oder verlinkt werden, statt mehrfach in Fließtext aufzutreten.

Darüber hinaus sollten Sprache und Struktur vereinheitlicht werden. Für technische Dokumentation bietet sich eine einheitliche Leitsprache an, während kunden- oder supportorientierte Inhalte bei Bedarf weiterhin deutschsprachig bleiben können. Dubletten wie mehrfach vorhandene Feature-Matrizen sollten zusammengeführt werden. Eine zentrale Einstiegsseite sollte die wichtigsten Dokumente, Zielgruppen und Ablageorte übersichtlich verknüpfen.

Die Dokumentationspflege sollte organisatorisch verankert werden. Relevante Aktualisierungen können als Bestandteil der Definition of Done behandelt werden. Klare Seitenverantwortliche und regelmäßige Review- und Archivierungsintervalle verhindern, dass unvollständige oder veraltete Inhalte dauerhaft in der aktiven Struktur verbleiben. Concurrent Documentation sollte konsequent abgeschlossen werden; offene Platzhalter sind entweder zu ergänzen oder sichtbar als Entwurf zu kennzeichnen.

Für wichtige technische Entscheidungen bieten sich leichtgewichtige Architecture Decision Records an. Dadurch können nicht nur Ergebnisse, sondern auch Ausgangslage, Begründung und verworfene Alternativen nachvollzogen werden. Dies ist insbesondere bei Taxonomieumstellungen, Berechnungsmechanismen und externen Schnittstellen sinnvoll.

Bei begrenzten Ressourcen sollte die Dokumentation risikobasiert priorisiert werden. Vorrang besitzen Zentralbank-Schnittstellen, Taxonomie- und Importprozesse sowie Berechnungs- und Validierungslogik, da diese Bereiche betriebs- und fachkritisch sind und sich nur mit hohem Aufwand aus dem Code rekonstruieren lassen. Finrep benötigt somit nicht grundsätzlich mehr Dokumentation, sondern eine konsistentere, leichter auffindbare und dauerhaft aktualisierbare Dokumentationsbasis.

5 Fazit

5.1 Beantwortung der Forschungsfrage

In agilen Softwareprojekten ist so viel Dokumentation sinnvoll, wie zur Sicherung des wesentlichen Projektwissens und zur Unterstützung von Entwicklung, Wartung und Betrieb erforderlich ist. Unnötige, redundante oder nicht dauerhaft pflegbare Inhalte sollten vermieden werden. Ein fester zeitlicher Anteil ist nicht zweckmäßig; der Aufwand muss abhängig von Komplexität, Kritikalität, Lebensdauer, Teamstruktur und externen Abhängigkeiten bestimmt werden.

Für Finrep ist der bestehende Umfang aufgrund der fachlichen und technischen Komplexität grundsätzlich angemessen. Verbesserungsbedarf besteht weniger bei der Menge als bei der Qualität, insbesondere bei der In-Code-Dokumentation, der langfristigen Aktualität sowie einer einheitlichen Sprache und Struktur. Durch risikobasierte Priorisierung und die Einbindung der Pflege in den Entwicklungsprozess lässt sich Projektwissen sichern, ohne die agile Weiterentwicklung unverhältnismäßig zu belasten.

5.2 Ausblick

Die empfohlenen Maßnahmen sollten schrittweise in den Entwicklungsprozess integriert werden. Priorität besitzen die Dokumentation kritischer Codebereiche, klare Verantwortlichkeiten und die automatische Aktualisierung technischer Übersichten. Anschließend kann untersucht werden, ob sich dadurch Einarbeitung, Wartung, Fehleranalyse und die Abhängigkeit von einzelnen Wissensträgern messbar verbessern.

Bibliographie

- [1] P. Abrahamsson, O. Salo, J. Ronkainen, und J. Warsta, „Agile Software Development Methods: Review and Analysis“, *Proc. Espoo 2002*, S. 3–107, Jan. 2002, doi: <https://doi.org/10.48550/arXiv.1709.08439>.
- [2] T. Dybå und T. Dingsøyr, „Empirical studies of agile software development: A systematic review“, *Information and Software Technology*, Bd. 50, Nr. 9, S. 833–859, 2008, doi: <https://doi.org/10.1016/j.infsof.2008.01.006>.
- [3] K. Conboy, „Agility from First Principles: Reconstructing the Concept of Agility in Information Systems Development“, *Information Systems Research*, Bd. 20, Nr. 3, S. 329–354, 2009, doi: [10.1287/isre.1090.0236](https://doi.org/10.1287/isre.1090.0236).
- [4] N. B. Moe, T. Dingsøyr, und T. Dybå, „A teamwork model for understanding an agile team: A case study of a Scrum project“, *Information and Software Technology*, Bd. 52, Nr. 5, S. 480–491, 2010, doi: <https://doi.org/10.1016/j.infsof.2009.11.004>.
- [5] B. Boehm und R. Turner, „Management challenges to implementing agile processes in traditional development organizations“, *IEEE Software*, Bd. 22, Nr. 5, S. 30–39, 2005, doi: <https://doi.org/10.1109/MS.2005.129>.
- [6] T. Lethbridge, J. Singer, und A. Forward, „How software engineers use documentation: the state of the practice“, *IEEE Software*, Bd. 20, Nr. 6, S. 35–39, 2003, doi: <https://doi.org/10.1109/MS.2003.1241364>.
- [7] A. Forward und T. C. Lethbridge, „The relevance of software documentation, tools and technologies: a survey“, in *Proceedings of the 2002 ACM Symposium on Document Engineering*, in DocEng '02. McLean, Virginia, USA: Association for Computing Machinery, 2002, S. 26–33. doi: <https://doi.org/10.1145/585058.585065>.
- [8] C. Treude, J. Middleton, und T. Atapattu, „Beyond accuracy: assessing software documentation quality“, in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in ESEC/FSE 2020. Virtual Event, USA: Association for Computing Machinery, 2020, S. 1509–1512. doi: <https://doi.org/10.1145/3368089.3417045>.
- [9] F. Paetsch, A. Eberlein, und F. Maurer, „Requirements engineering and agile software development“, in *WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, 2003.*, 2003, S. 308–313. doi: <https://doi.org/10.1109/ENABL.2003.1231428>.
- [10] V. T. Heikkilä, D. Damian, C. Lassenius, und M. Paasivaara, „A Mapping Study on Requirements Engineering in Agile Software Development“, in *2015 41st Euromicro Conference on Software Engineering and Advanced Applications*, 2015, S. 199–207. doi: <https://doi.org/10.1109/SEAA.2015.70>.
- [11] S. C. B. de Souza, N. Anquetil, und K. M. de Oliveira, „A study of the documentation essential to software maintenance“, in *Proceedings of the 23rd Annual International Conference on Design of Communication: Documenting & Designing for Pervasive Information*, in SIGDOC '05. Coventry, United Kingdom: Association for Computing Machinery, 2005, S. 68–75. doi: <https://doi.org/10.1145/1085313.1085331>.
- [12] C. J. Stettina und W. Heijstek, „Necessary and neglected? an empirical study of internal documentation in agile software development teams“, in *Proceedings of the 29th ACM International Conference on Design of Communication*, in SIGDOC '11. Pisa, Italy: Association for Computing Machinery, 2011, S. 159–166. doi: <https://doi.org/10.1145/2038476.2038509>.
- [13] E. Bjarnason, K. Wnuk, und B. Regnell, „Requirements are slipping through the gaps — A case study on causes & effects of communication gaps in large-scale software development“, in *2011 IEEE 19th*

- International Requirements Engineering Conference*, 2011, S. 37–46. doi: <https://doi.org/10.1109/RE.2011.6051639>.
- [14] B. Ramesh, L. Cao, und R. Baskerville, „Agile requirements engineering practices and challenges: an empirical study“, *Information Systems Journal*, Bd. 20, Nr. 5, S. 449–480, 2010, doi: <https://doi.org/10.1111/j.1365-2575.2007.00259.x>.
- [15] I. Inayat, S. S. Salim, S. Marczak, M. Daneva, und S. Shamshirband, „A systematic literature review on agile requirements engineering practices and challenges“, *Computers in Human Behavior*, Bd. 51, S. 915–929, 2015, doi: <https://doi.org/10.1016/j.chb.2014.10.046>.
- [16] R. Kasauli, E. Knauss, J. Horkoff, G. Liebel, und F. G. de Oliveira Neto, „Requirements engineering challenges and practices in large-scale agile system development“, *Journal of Systems and Software*, Bd. 172, S. 110851, 2021, doi: <https://doi.org/10.1016/j.jss.2020.110851>.
- [17] M. A. Islam, R. Hasan, und N. U. Eisty, „Documentation Practices in Agile Software Development: A Systematic Literature Review“, in *2023 IEEE/ACIS 21st International Conference on Software Engineering Research, Management and Applications (SERA)*, 2023, S. 266–273. doi: <https://doi.org/10.1109/SERA57763.2023.10197828>.